ZaraOS

FULL IMPLEMENTATION REPORT · ALPHA 0.4

Generated May 11, 2026

Status: Published & Live

Briefing for ChatGPT — Complete context to resume work on ZaraOS

ZaraOS is a futuristic AI-native desktop operating environment prototype. It runs entirely in the browser (React + Vite), has no backend dependencies for Alpha 0.4, and is designed to be packaged as a Tauri desktop app and eventually a bootable Linux ISO. The core philosophy is local-first AI: all inference runs on-device by default; cloud providers are opt-in with the user's own API keys. This document captures every system built, all architectural decisions, the complete AI provider stack, and the roadmap.

Contents

1. PROJECT OVERVIEW & CORE CONSTRAINTS

ZaraOS is a web-based prototype of an AI-first operating environment. It runs entirely in the browser (React + Vite), has no backend dependencies for Alpha 0.4, and is designed to be packaged later as a Tauri desktop app and eventually a bootable Linux ISO.

Vision

- **Short term** — Feature-complete browser prototype with real AI inference, voice, and gesture
- **Medium term** — Tauri-packaged desktop app (Windows / macOS / Linux) with Whisper.cpp offline voice
- **Long term** — Bootable ZaraOS Linux ISO with Ollama pre-installed, Zara as the shell

Core Constraints — Never Break These

- **No emojis** — in the UI ever, no exceptions
- **Always dark mode** — no light mode toggle at the OS level
- **Deny-by-default** — mic, camera, cloud AI, network, files — all off at launch
- **UI !' Runtime only** — UI calls useRuntime(), never calls AI/voice/gesture engines directly
- **Local AI default** — cloud AI is opt-in, user-provided keys, ZaraOS pays nothing for inference
- **Transparent AI state** — Zara always tells the user when she is in simulated mode vs real AI

2. TECH STACK & MONOREPO STRUCTURE

Stack

- **Package manager** — pnpm workspaces (monorepo) — workspace packages use @workspace/ prefix
- **Runtime** — Node.js 24, TypeScript 5.9 strict mode throughout
- **Frontend** — React + Vite — code-split with React.lazy() + Suspense on all 11 pages
- **Styling** — Tailwind CSS + shadcn/ui component library
- **Animation** — framer-motion — used for VoiceWaveform and panel transitions
- **Routing** — wouter (lightweight client-side router)
- **State** — React Context + localStorage — no backend for Alpha 0.4
- **API Server** — Express 5 artifact exists but is NOT yet called by ZaraOS
- **Database** — PostgreSQL + Drizzle ORM schema exists but is NOT yet wired
- **Build** — esbuild for API server CJS bundle; Vite for frontend

Monorepo layout

```
artifacts/zaraos/        !• ZaraOS frontend (this is the main product)
artifacts/api-server/     !• Express 5 API server (Alpha 0.5+)
artifacts/mockup-sandbox/ !• Canvas design sandbox (internal tooling)
lib/api-spec/             !• OpenAPI contract + React Query codegen
lib/db/                   !• Drizzle ORM schema
scripts/                  !• Utility scripts (this file)
```

Key commands

- **pnpm --filter @workspace/zaraos run dev** — Start the ZaraOS frontend
- **pnpm --filter @workspace/zaraos run typecheck** — TypeScript check (always run after changes)
- **pnpm run typecheck** — Full workspace typecheck including libs
- **pnpm --filter @workspace/api-spec run codegen** — Regenerate API hooks from OpenAPI spec

3. ARCHITECTURE — COMMAND FLOW

Every user input — voice, gesture, keyboard, or plugin — flows through the same layered pipeline. The UI calls useRuntime() exclusively. No component touches an AI provider or hardware engine directly.

```
UI (useRuntime hook)
  % %  ZaraRuntime.executeCommand(input, source)
       % %  parseAndRoute(input, source) !’ ParsedCommand
       %      { raw, normalized, intent, target, skillId,
       %        source, requiresPermission, destructive, confidence }
       %
       % %  intent === "skill_action"  !’ SkillRuntime.executeSkill()
       % %  requiresPermission check  !’ permission_denied gate
       % %  destructive check         !’ confirm_required gate
       % %  intent === "ai_question"  !’ streamAssistantMessage()
       %      % %  aiRuntime.streamMessage(msg, onChunk, source, intent)
       %            !’ buildRequestPayload(msg, intent)
       %                !’ buildSystemPrompt(undefined, intent)   !• intent-aware
       %                !’ buildContextBlock(injectionInput)      !• live OS state
       %                !’ conversationMemory.getMessagesForContext(3000)
       %            !’ requestRouter.dispatch()
       %                !’ providerRouter.route()                 !• cached health
       %                !’ modelRouter.selectModel()
       %                !’ provider.streamMessage()
       % %  all other intents         !’ CommandResult { action, payload }
```

Key TypeScript types (src/core/types.ts)

- **CommandIntent** — open_app | close_app | navigation_action | scroll_action | search | file_action | media_action | ai_question | system_status | privacy_action | settings_action | developer_action | skill_action | unknown
- **ZaraStatus** — idle | listening | thinking | speaking | offline | privacy-lock
- **InputSource** — voice | gesture | keyboard | system | plugin
- **ParsedCommand** — raw, normalized, intent, target, skillId, source, requiresPermission, destructive, confidence
- **CommandResult** — success, intent, response, action, payload, skillId, requiresConfirmation, dangerous, confirmationReason, source, timestamp
- **PermissionCategory** — microphone | camera | cloud_ai | network | files | system_actions | developer_mode | plugins | local_ai

4. AI PROVIDER STACK (SRC/CORE/AI/)

The AI layer is fully layered: AIRuntime !’ RequestRouter !’ ProviderRouter !’ Provider. No component in the UI touches a provider directly. All 6 providers implement the same AIProviderAdapter interface.

Provider priority — local_first strategy (default)

Zara Local Runtime (local) [Always] Simulated responses. Guaranteed fallback. Always registered.

Ollama (ollama) [If running] Real local LLM at localhost:11434. NDJSON streaming. /api/version health check.

llama.cpp Server (llamacpp) [If running] Real local LLM at localhost:8080. OpenAI-compatible REST API.

OpenAI (openai) [Opt-in] SSE streaming to api.openai.com. /v1/models health check. Default: gpt-4o-mini.

Anthropic (anthropic) [Opt-in] SSE streaming. anthropic-dangerous-direct-browser-access header required. Default: claude-3-5-haiku-latest.

Google Gemini (gemini) [Opt-in] SSE via ?alt=sse. API key as URL query param. /v1beta/models health check. Default: gemini-1.5-flash.

Provider routing logic

When preferredProviderId is set, the router tries that provider first and falls back on failure. In local_first mode: tries Ollama !' llama.cpp !' local. If cloudAIEnabled=true, tries cloud providers as a last resort before the guaranteed local fallback.

CORS details per cloud provider

- **OpenAI** — Natively allows browser CORS. Standard Authorization: Bearer {key} header.
- **Anthropic** — Requires 'anthropic-dangerous-direct-browser-access: true' header on every request. Official opt-in.
- **Gemini** — No auth header used. API key passed as ?key=... URL query param. Google permits browser CORS.

Provider Adapter Interface

```
interface AIProviderAdapter {
  id: string;  name: string;  isLocal: boolean;  isCloud: boolean;  isEnabled: boolean;
  initialize(): Promise<void>;
  sendMessage(messages: AIMessage[], options?: AISendOptions): Promise<string>;
  streamMessage(messages: AIMessage[], onChunk: AIStreamCallback, options?: AISendOptions): Promise<void>;
  healthCheck(): Promise<AIProviderStatus>;
  listModels(): Promise<string[]>;
  getActiveModel(): string;  setModel(modelId: string): void;
  supportsStreaming(): boolean;  supportsVision(): boolean;
  supportsTools(): boolean;     supportsOffline(): boolean;
}
```

Cloud AI Gate

ProviderRouter.cloudAIEnabled is false at startup. It is set to true only when the user grants the cloud_ai permission in Privacy settings. Wired via: ZaraRuntime.initialize() !' setCloudAIAllowed() and ZaraRuntime.requestPermission('cloud_ai') / revokePermission('cloud_ai'). This was a critical bug fixed in Alpha 0.4 — the gate was never being set before.

5. HEALTH CHECK CACHING (PROVIDERROUTER)

Without caching, every message dispatch triggered a healthCheck() on Ollama and llama.cpp — each with a 2-3 second network timeout when those servers are not running. This created a 3+ second penalty before every AI response.

Cache policy

- **available=true** — Cache for 60 seconds — re-validates healthy providers once per minute
- **available=false** — Cache for 20 seconds — allows quick detection when a local server starts
- **Cache bypass** — invalidateHealthCache(id) is called before explicit UI 'Test' button clicks
- **Auto-eviction** — Cache cleared on: API key change, endpoint change, provider re-registration
- **getCachedStatus(id)** — Peek at cache without triggering a live check — used by the UI status display

Implementation

```
// ProviderRouter.cachedHealthCheck() — called by route() on every dispatch
private async cachedHealthCheck(provider): Promise<AIProviderStatus> {
  const cached = this.healthCache.get(provider.id);
  if (cached && Date.now() < cached.expiresAt) return cached.status;   // HIT
  const status = await provider.healthCheck();                          // MISS !' live
  const ttl = status.available ? 60_000 : 20_000;
  this.healthCache.set(provider.id, { status, expiresAt: Date.now() + ttl });
  return status;
}
```

6. SYSTEM PROMPT SYSTEM (SRC/CORE/AI/PROMPTS/ZARA-SYSTEM-PROMPT.TS)

The system prompt is composed dynamically per request. When a concrete intent is known, a per-intent addendum replaces the generic ZARA_COMMAND_PARSING section. This allows Zara to respond with the exact behavior appropriate for the command class.

Composition order

```
1. ZARA_BASE_SYSTEM_PROMPT       — identity, personality, voice & tone, what Zara is not
2. ZARA_PRIVACY_PRINCIPLES        — 6 local-first privacy principles
3. ZARA_LOCAL_FIRST_PHILOSOPHY    — local AI default, cloud opt-in, BYOK model
4. ZARA_CONFIRMATION_RULES        — destructive action gates
5. ZARA_SKILL_PHILOSOPHY          — skill routing transparency
6. ZARA_SAFETY_RULES              — permission enforcement rules
7. ZARA_INTENT_ADDENDUMS[intent]  — OR ZARA_COMMAND_PARSING (fallback)
8. CURRENT SYSTEM STATE block     — live OS context (provider, model, permissions, panel)
```

buildSystemPrompt(context?, intent?)

Selects the matching addendum when intent is a key of ZARA_INTENT_ADDENDUMS. Falls back to generic ZARA_COMMAND_PARSING when intent is undefined or unrecognized. The intent is already in scope in buildRequestPayload() via the intent parameter passed from aiRuntime.streamMessage().

Intent-Specific Addendums (14 entries)

• **ai_question**	Conversational tone, 1-4 sentences, no OS jargon, don't fabricate live data
• **search**	Lead with the answer, name ZaraOS location, suggest alternatives if missing
• **open_app**	One-sentence confirmation, permission check, closest match if app unknown
• **close_app**	One-sentence confirm, check for unsaved work before closing
• **navigation_action**	Single short confirm, don't re-describe the destination panel
• **scroll_action**	Acknowledge briefly or silently, note if no scrollable area present
• **file_action**	State action + target before executing, files permission gate, always confirm destructive ops
• **media_action**	Minimal acknowledgement, no preamble
• **system_status**	Data-driven short list from context block, no speculation, mark unknowns
• **privacy_action**	Announce what changes before applying, note mid-session interruptions
• **settings_action**	Confirm setting + new value in one line, note any side effects
• **developer_action**	Technical precision, surface security implications explicitly
• **skill_action**	Name the skill, state required permissions, explicit confirm request before executing
• **unknown**	No guessing, no action, ask for clarification with a specific suggestion

7. CONVERSATION MEMORY (SRC/CORE/AI/MEMORY/)

• **ConversationSession**	Started or resumed on aiRuntime.initialize(). Sessions resume if last activity < 30 min.
• **Message storage**	All messages in localStorage under zaraos_session_* keys. Restored on reload.
• **Context pruning**	getMessagesForContext(3000) — walks backwards, stops at ~3000 tokens (1 token "H 4 chars).
• **Memory entries**	Separate from messages — persistent facts, session facts, pinned entries. Survive session resets.
• **Skill usage tracking**	Every skill execution logged: skillId, lastUsedAt, useCount, lastResult.
• **User preferences**	Stored separately, survive all purge operations except purgeAll().
• **clearAllHistory()**	Wipes messages, preserves pinned entries.

- **purgeAll()** — Nuclear option — wipes everything including preferences.
- **estimateStorageBytes()** — Returns current localStorage usage estimate for memory stats display.

Storage keys

```
zaraos_session_current_id   !• current session ID
zaraos_session_{id}          !• full session object with messages array
zaraos_memory_entries        !• persistent/session/pinned MemoryEntry[]
zaraos_skill_usage           !• SkillUsageRecord[]
zaraos_preferences           !• UserPreferences object
```

8. VOICE ENGINE & WAVEFORM ANIMATION

VoiceEngine (src/lib/voice-engine.ts) — Alpha 0.4, fully wired

Real voice input via the Web Speech API. Alpha 0.5+ will replace with Whisper.cpp via Tauri subprocess. The engine interface will stay identical — only the internals change.

- **Browser support** — Chrome 33+, Edge 79+, Safari iOS 14+. Firefox: NOT supported.
- **States** — idle !' listening !' idle / error / unsupported
- **Interim results** — Streamed character-by-character to the input box as the user speaks
- **Final results** — Fired on utterance completion, routed to zaraRuntime.executeCommand()
- **Error handling** — 11 named error codes with user-facing messages. 'aborted' is silent.
- **simulateVoiceInput()** — Dev/testing helper — fires interim+final callbacks without a real mic
- **Alpha 0.5+ plan** — Whisper.cpp via Tauri: window.__TAURI__.invoke('transcribe') !' same onResult callbacks

VoiceWaveform (src/components/voice-waveform.tsx) — Alpha 0.4, new

7 framer-motion bars with staggered animation speeds and peak amplitudes. Collapses to a flat resting state (scaleY=0.12) when active=false. Three surface deployments:

- **Assistant page** — Replaces the pulsing amber dot above the input bar during LISTENING state
- **Global command box** — Replaces the pulsing dot in the header LISTENING badge
- **Sidebar Voice toggle** — Replaces the static active dot when voice mode is enabled

Props: active (bool), color ('amber' | 'cyan' | 'purple'), size ('xs' | 'sm' | 'md').

```
// 7 bars — each tuple: [peakScaleY, durationSeconds, delaySeconds]
const BAR_CONFIGS = [
  [0.35, 0.70, 0.00], [0.90, 0.52, 0.08], [0.60, 0.80, 0.16],
  [1.00, 0.46, 0.04], [0.55, 0.64, 0.12], [0.80, 0.56, 0.20], [0.30, 0.74, 0.06],
];
```

9. PERMISSIONS SYSTEM (SRC/CORE/PERMISSIONS.TS)

Deny-by-default. All permissions are OFF at launch. State persists to localStorage. The UI requests permissions through ZaraRuntime.requestPermission(category).

- **microphone** — Required for voice input (VoiceEngine.startListening())
- **camera** — Reserved for future vision / gesture features
- **cloud_ai** — Required for cloud providers. Also gates ProviderRouter.cloudAIEnabled.
- **network** — General network access for non-AI network calls
- **files** — File system read/write access
- **system_actions** — OS-level actions (shutdown, reboot, etc.)
- **developer_mode** — Plugin development, raw API access
- **plugins** — Third-party plugin installation and execution

- **local_ai** Local AI inference (Ollama, llama.cpp)

10. PAGES & PANELS (SRC/PAGES/)

• **home.tsx — Dashboard**	Live clock, CPU/RAM/Network/Neural stats, Privacy Fortress panel, System Activity feed
• **assistant.tsx — Assistant**	Full AI chat — SSE streaming, ZaraStatus states, voice input with interim transcript, memory stats, VoiceWaveform, conversation clear
• **console.tsx — Console**	Natural language command console, intent routing, command history, source badges (voice/gesture/keyboard)
• **apps.tsx — App Launcher**	App grid with voice command hints per tile. Launches via zaraRuntime.launchApp()
• **files.tsx — Files**	Placeholder file browser — permission-gated behind files permission
• **media.tsx — Media**	Combined audio/video player placeholder
• **settings.tsx — Settings**	System configuration, Input Mode tab, Gestures tab with test buttons
• **privacy.tsx — Privacy**	Mic/camera/AI/network/files status and enable/disable toggles — directly calls requestPermission()
• **ai-providers.tsx — AI Providers**	Full provider manager: enable/disable, API key entry, endpoint config, Test button, preferred provider pin, health status display
• **developers.tsx — Developers**	Plugin registry, PluginManifest spec, 4 example plugins, Zara Store preview
• **skills.tsx — Skills**	Skill hub — lists all registered skills with permission status, usage stats, enable/disable
• **not-found.tsx**	404 fallback page

11. GLOBAL COMMAND BOX (SRC/COMPONENTS/GLOBAL-COMMAND-BOX.TSX)

A Spotlight/Alfred-style overlay accessible from anywhere via Ctrl+Space. Supports voice and keyboard input. Routes all commands through zaraRuntime.executeCommand() — same pipeline as everything else.

• **Trigger**	Ctrl+Space global keyboard shortcut (registered once in InputModeProvider)
• **Voice button**	Inside the box — same VoiceEngine as the assistant page
• **LISTENING badge**	Shows VoiceWaveform (xs, amber) + 'LISTENING' when voice is active in the box
• **Command routing**	All commands: zaraRuntime.executeCommand(input, 'voice' | 'keyboard')
• **Auto-navigation**	If result.action === 'navigate', box closes and wouter.navigate() fires
• **History**	Up/Down arrows navigate last 5 commands. Shown as clickable chips.
• **Quick suggestions**	6 suggestion chips pre-populated with common commands

12. COMPLETED IMPLEMENTATION ITEMS

Item 1 **Ollama Provider Routing** [DONE]

- Real HTTP POST to localhost:11434/api/chat (OpenAI-compatible format)
- NDJSON streaming: each line is JSON with { message: { content }, done }
- healthCheck() via GET /api/version — 3 second timeout
- Graceful fallback to simulated responses if Ollama is not running
- Router priority: Ollama !' llama.cpp !' local (in local_first strategy)
- Tauri plan: Rust backend will manage Ollama process via std::process::Command

---

Item 2 **Web Speech API Voice Input** [DONE]

- Real voice input via SpeechRecognition / webkitSpeechRecognition
- Interim results (isFinal=false) stream to the input box character by character
- Final result (isFinal=true) fires zaraRuntime.executeCommand(transcript, 'voice')
- 11 named error codes with user-facing messages. 'aborted' is silent.
- Firefox unsupported notice displayed when SpeechRecognition is unavailable
- Wired into assistant.tsx and global-command-box.tsx

Item 3 Route-Based Code Splitting [DONE]

- All 11 page components wrapped in React.lazy() + Suspense
- Initial bundle loads only the OS shell (layout, sidebar, routing)
- Each panel chunk loads on first navigation to that route
- Loading skeleton displayed during chunk fetch
- Vite automatically splits at dynamic import boundaries

Item 4 Real Cloud Provider HTTP Inference [DONE]

- OpenAI: SSE streaming to api.openai.com/v1/chat/completions (stream: true)
- OpenAI: healthCheck via GET /v1/models — validates key without token use
- Anthropic: SSE streaming with 'anthropic-dangerous-direct-browser-access: true' header
- Anthropic: system prompt via top-level 'system' field (not in messages array)
- Anthropic: healthCheck via GET /v1/models
- Gemini: SSE via ?alt=sse to /v1beta/models/{model}:streamGenerateContent
- Gemini: API key as ?key= URL param, 'model' role for assistant messages
- Gemini: healthCheck via GET /v1beta/models?key=...
- BUG FIX: providerRouter.cloudAIEnabled was always false — wired setCloudAIAllowed()
- through ZaraRuntime.initialize() / requestPermission() / revokePermission()

Item 5 Health Check Caching [DONE]

- ProviderRouter wraps all healthCheck() calls in cachedHealthCheck()
- TTL: 60s for available providers, 20s for unavailable
- Cache evicted on: API key change, endpoint change, re-registration, UI 'Test' click
- Eliminates 2-3 second network timeout penalty on every message dispatch
- getCachedStatus(id) for UI to peek without triggering a live check
- invalidateHealthCache(id?) for selective or full eviction

Item 6 Voice Waveform Animation [DONE]

- VoiceWaveform: 7 framer-motion bars, staggered amplitudes (0.30–1.00) and durations (0.46–0.80s)
- Collapses to scaleY=0.12 flat resting state when active=false
- Props: active (bool), color ('amber'|'cyan'|'purple'), size ('xs'|'sm'|'md')
- Deployed in: assistant.tsx listening bar, global-command-box.tsx header, sidebar Voice toggle
- Replaces static pulsing amber dots in all three surfaces

Item 7 Intent-Aware System Prompts [DONE]

- ZARA_INTENT_ADDENDUMS: 14 intent-specific behavioral sections
- buildSystemPrompt(context?, intent?) selects matching addendum when intent is known
- Falls back to generic ZARA_COMMAND_PARSING when intent is undefined/unknown
- Intent is already in buildRequestPayload() scope — one-line wire-up
- Each addendum tells Zara exactly what tone, format, and constraints apply

13. UPCOMING ROADMAP (NEXT ITEMS)

• **Item 8: Streaming in Console**	Real-time token streaming in the Console panel (currently non-streaming)
• **Item 9: Skill execution with real AI**	Skill router passes context to AI; AI generates skill-specific responses
• **Item 10: Memory panel UI**	Expose conversation memory stats, pinned entries, and purge controls in the UI
• **Item 11: Tauri packaging**	Wrap Vite app in Tauri shell for native desktop distribution
• **Item 12: Whisper.cpp offline voice**	Replace Web Speech API with Whisper.cpp via Tauri subprocess
• **Item 13: Ollama model selector**	Live model list from /api/tags, model download progress UI, active model indicator
• **Item 14: Encrypted key storage**	Move API keys from localStorage to encrypted IndexedDB (Web Crypto AES-GCM)
• **Item 15: Linux ISO build**	Archiso / Debian live-build with ZaraOS as WM + Ollama as systemd service

14. KEY FILE MAP (ARTIFACTS/ZARAOS/SRC/)

```
core/
  types.ts                          !• ALL shared TypeScript interfaces
  zara-runtime.ts                   !• OS brain — single entry point for all commands
  runtime-context.tsx               !• useRuntime() React hook + RuntimeProvider
  permissions.ts                    !• Deny-by-default PermissionsManager singleton
  input-mode.tsx                    !• InputModeContext — voice/gesture/mode state
  ai/
    ai-runtime.ts                   !• Central AI orchestrator (AIRuntime singleton)
    prompts/
      zara-system-prompt.ts         !• buildSystemPrompt(), ZARA_INTENT_ADDENDUMS, simulated responses
    providers/
      provider-adapter.ts           !• AIProviderAdapter interface + AIStreamCallback type
      provider-registry.ts          !• All provider instances, localStorage persistence, cloud gate
      local-provider.ts             !• Simulated fallback (always available, zero latency)
      ollama-provider.ts            !• Ollama NDJSON streaming, /api/version health check
      llamacpp-provider.ts          !• llama.cpp OpenAI-compat REST
      openai-provider.ts            !• OpenAI SSE, /v1/models health check
      anthropic-provider.ts         !• Anthropic SSE, dangerous-direct-browser-access
      gemini-provider.ts            !• Gemini SSE alt=sse, key as query param
    routing/
      provider-router.ts            !• Routing strategy + health check cache (60s/20s TTL)
      request-router.ts             !• Provider + model selection + dispatch
      model-router.ts               !• Classifies request type !’ selects best model
    memory/
      conversation-memory.ts        !• Session, message, entry, skill usage manager
      memory-storage.ts             !• localStorage persistence layer
      memory-types.ts               !• ConversationMessage, MemoryEntry, MemoryStats, etc.
    context/
      context-injector.ts           !• buildContextBlock() — assembles live OS state string
      system-context.ts             !• SystemContextSnapshot
      privacy-context.ts            !• PrivacyContextSnapshot
      skills-context.ts             !• SkillContextEntry[]
    models/
      ai-capabilities.ts            !• Provider capability constants (OPENAI_CAPABILITIES, etc.)
      model-router.ts               !• Request classification !’ model selection
  skills/
    skill-runtime.ts                !• Skill execution engine
    types.ts                        !• ZaraSkill, SkillExecutionResult interfaces

lib/
  voice-engine.ts                   !• Web Speech API voice input singleton
  gesture-engine.ts                 !• MediaPipe Hands placeholder + simulation
  gesture-mapper.ts                 !• GestureType !’ command string mapping
  command-router.ts                 !• parseAndRoute() — NLP intent classification
  ai-engine.ts                      !• Legacy shim (deprecated, use ai-runtime directly)
  privacy-store.ts                  !• usePrivacy() hook

components/
  layout.tsx                        !• Desktop OS shell (sidebar + main panel frame)
  global-command-box.tsx            !• Ctrl+Space overlay with voice + VoiceWaveform
  voice-waveform.tsx                !• 7-bar framer-motion waveform component
  ai-runtime-status.tsx             !• Provider/model/latency status badge
  input-mode-indicator.tsx          !• Voice/Gesture/Keyboard mode display + switcher
  confirmation-dialog.tsx           !• Destructive action confirm modal
  error-boundary.tsx                !• React error boundary wrapper

pages/
  home.tsx          assistant.tsx   console.tsx      apps.tsx
  files.tsx         media.tsx       settings.tsx     privacy.tsx
```

15. IMPORTANT GOTCHAS & DESIGN DECISIONS

• cloudAIEnabled bug (fixed)	ProviderRouter.cloudAIEnabled was always false at startup — wired setCloudAIAllowed() through ZaraRuntime.initialize() / requestPermission('cloud_ai') / revokePermission('cloud_ai').
• Anthropic CORS header	'anthropic-dangerous-direct-browser-access: true' is required on every request. Official Anthropic opt-in for browsers.
• Gemini key in URL	API key is a URL query param (?key=...). Only safe over HTTPS. Key appears in request logs.
• Ollama healthCheck endpoint	/api/version (not /api/tags or /v1/models). Any 200 response = available.
• API keys in localStorage	Intentional Alpha 0.4 prototype behavior, clearly labeled in the UI. Encrypted IndexedDB planned for Alpha 0.4.
• No root pnpm dev	No root dev script by design. Artifacts start via their own workflows with PORT and BASE_PATH. Never run 'pnpm dev' at workspace root.
• Typecheck command	Always run 'pnpm --filter @workspace/zaraos run typecheck' after making changes. Zero errors is the bar.
• App self-import (fixed in Alpha 0.1)	Original scaffold had a circular self-import in App.tsx. Fixed: exports MainApp default wrapped in RuntimeProvider.
• Anthropic messages array	Anthropic does not accept a 'system' role in the messages array. System prompt is a top-level 'system' field.
• Gemini role convention	Gemini uses 'model' for assistant role (not 'assistant'). The adapter converts this transparently.